



Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Selection sort

44	11	55	22	66	33
0	1	2	3	4	5

i	minIndex	j
0	0	1
	1	2
		3
		4
		5
		6X

```

minIndex = i
for (j = i + 1; j < N; j++)
    if (arr[j] < arr[minIndex])
        minIndex = j;
    
```

11	44	55	22	66	33
0	1	2	3	4	5

i	minIndex	j
1	1	2
	2	3
		4
		5
		6X

$$S(n) = O(n^2)$$

n - no. of elements	passes	comps
passes = n - 1	1	n - 1
	2	n - 2
	3	n - 3
	⋮	⋮
	⋮	2
	n - 1	1

$$\begin{aligned} \text{Total comps} &= 1 + 2 + 3 + \dots + (n - 2) + (n - 1) \\ &= \frac{n(n + 1)}{2} \end{aligned}$$

$$\text{Time} \propto \frac{1}{2}(n^2 + n)$$

$$T(n) = O(n^2)$$

Best
Avg
worst

Mathematical polynomials:

$$f(n) = n^2 + n$$

where n - variable

Degree - highest power of a variable in polynomial.

- while writing time & space complexities consider only degree term, because that term is highest growing term in the polynomial.

n	n^2
1	1
10	100
100	10000
1000	1000000

Bubble sort

1. Compare all pairs of consecutive elements of the array one by one
2. If left element is greater than right element , then swap both
3. Repeat above steps until array is sorted $(N-1)$

11 22 33 44 55 66
 11 22 33 44 55 66
 11 22 33 44 55 66
 11 22 33 44 55 66
 11 22 33 44 55 66

Comps = $n-1$

Time $\propto n-1$

$$T(n) = O(n) \text{ Best}$$

Pass	Comps
1	$n-1$
2	$n-2$
$\vdots$	$\vdots$
$n-2$	2
$n-1$	1

No. of elements = n

No. of passes = $n-1$

$$\begin{aligned}
 \text{Total comps} &= 1 + 2 + 3 + \dots + (n-2) + (n-1) \\
 &= \frac{n(n+1)}{2} \\
 &= \frac{1}{2}(n^2 + n)
 \end{aligned}$$

Time $\propto$ comps

$$\text{Time} \propto \frac{1}{2}(n^2 + n)$$

$$T(n) = O(n^2) \text{ Avg worst}$$

Bubble sort

33	22	66	55	44	11
0	1	2	3	4	5

j $j+1$

33	22	66	55	44	11
0	1	2	3	4	5
22	33	66	55	44	11
0	1	2	3	4	5
22	33	66	55	44	11
0	1	2	3	4	5
22	33	55	66	44	11
0	1	2	3	4	5
22	33	55	44	66	11
0	1	2	3	4	5
22	33	55	44	11	66
0	1	2	3	4	5

22	33	55	44	11	66
0	1	2	3	4	5
22	33	55	44	11	66
0	1	2	3	4	5
22	33	55	44	11	66
0	1	2	3	4	5
22	33	44	55	11	66
0	1	2	3	4	5
22	33	44	11	55	66
0	1	2	3	4	5
22	33	44	11	55	66
0	1	2	3	4	5

22	33	44	11	55	66
0	1	2	3	4	5
22	33	44	11	55	66
0	1	2	3	4	5
22	33	44	11	55	66
0	1	2	3	4	5
22	33	11	44	55	66
0	1	2	3	4	5
22	33	11	44	55	66
0	1	2	3	4	5
22	33	11	44	55	66
0	1	2	3	4	5

22	33	11	44	55	66
0	1	2	3	4	5
22	33	11	44	55	66
0	1	2	3	4	5
22	11	33	44	55	66
0	1	2	3	4	5

22	11	33	44	55	66
0	1	2	3	4	5
11	22	33	44	55	66
0	1	2	3	4	5

to count passes : $i = 1$ to $N-1$ ($i < N$)
to do comps : $j = 0$ to $N-2$ ($j < N-1$)

i	$j < N-1$	$j < N-i$
1	$0 \rightarrow 4$	$0 \rightarrow 4$ ($j < 5$)
2	$0 \rightarrow 4$	$0 \rightarrow 3$ ($j < 4$)
3	$0 \rightarrow 4$	$0 \rightarrow 2$ ($j < 3$)
4	$0 \rightarrow 4$	$0 \rightarrow 1$ ($j < 2$)
5	$0 \rightarrow 4$	$0 \rightarrow 0$ ($j < 1$)

Insertion sort

1. Pick one element of the array (start from 2nd element)
2. Compare picked element with all its left neighbours one by one
3. If left neighbour is greater, move it one position ahead
4. Insert picked element at its appropriate position
5. Repeat above steps until array is sorted

pass 1 : 11 22 33 44 55 66
 pass 2 : 11 22 33 44 55 66
 pass 3 : 11 22 33 44 55 66
 pass 4 : 11 22 33 44 55 66
 pass 5 : 11 22 33 44 55 66

No. of comps = $n-1$

Time $\propto n-1$

$T(n) = O(n)$ Best

	passes	comps
No. of elements = n	1	1
No. of passes = $n-1$	2	2
	3	3
	$\vdots$	
	$n-1$	$n-1$

Total comps = $1+2+3+\dots+n$
 $= \frac{n(n+1)}{2}$

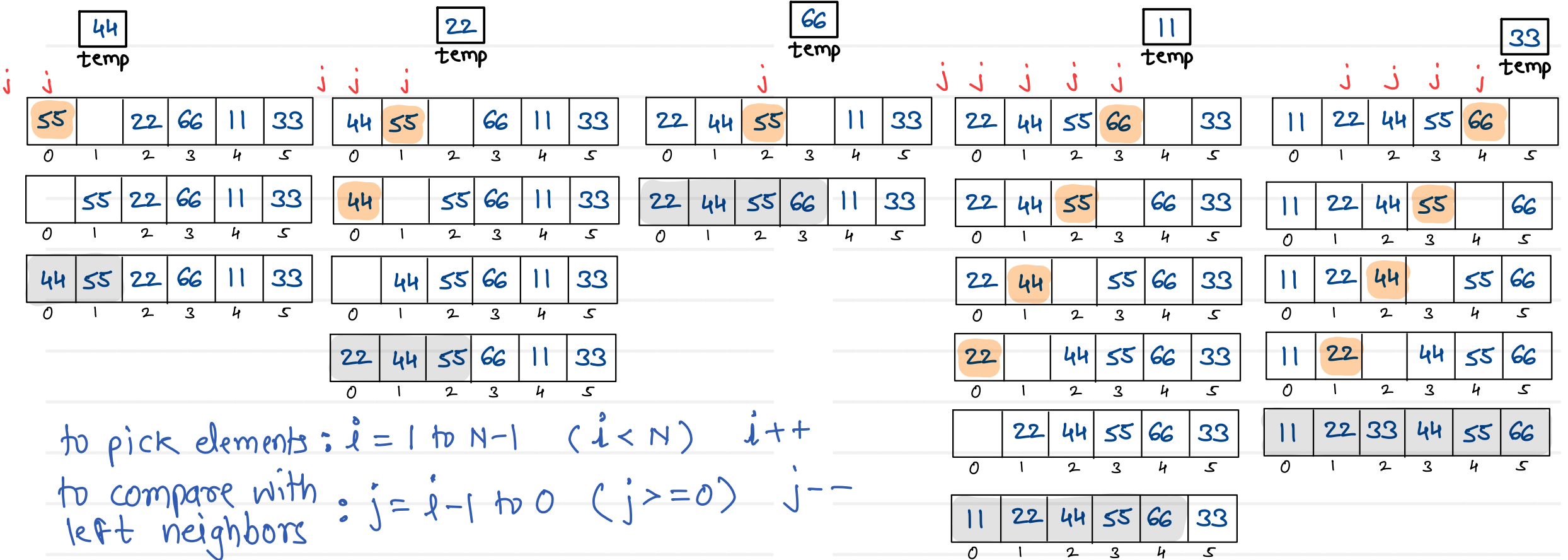
Time $\propto \frac{1}{2}(n^2+n)$

$T(n) = O(n^2)$ Worst Avg

$S(n) = O(1)$

Insertion sort

55	44	22	66	11	33
0	1	2	3	4	5



Insertion sort

```

for( i=1 ; i<N ; i++) {
    temp = arr[i]
    j;
    for( j=i-1 ; j>=0 ; j--) {
        if( arr[j] > temp )
            arr[j+1] = arr[j];
        else
            break;
    }
    arr[j+1] = temp;
}

```

~

11	22	33	44	55	66
----	----	----	----	----	----

0 1 2 3 4 5

i	i < 6	temp	j
1	T	44	0, -1
2	T	22	1, 0, -1
3	T	66	2
4	T	11	3, 2, 1, 0, -1
5	T	33	4, 3, 2, 1



Merge sort

1. Divide array in two parts
2. Sort both partitions individually (by merge sort only)
3. Merge sorted partitions into temporary array
4. Overwrite temporary array into original array

levels of division = $\log n$
comps per level $\approx n$

Total comps = $n \log n$
Time $\propto n \log n$

$T(n) = O(n \log n)$ Best Avg worst



$$\text{mid} = \frac{\text{left} + \text{right}}{2}$$

left partition: left to mid
right partition: mid+1 to right

Auxiliary space contains temp array of size equal to array size

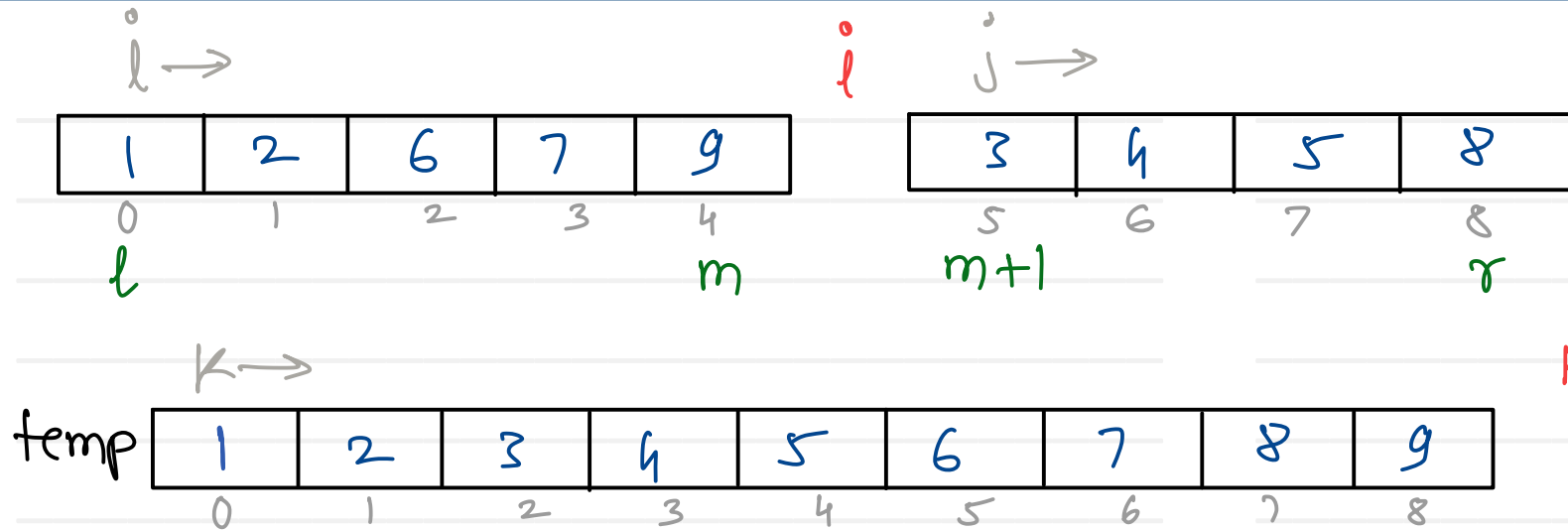
Aux space = n

Aux space $\propto n$

$$S(n) = O(n)$$



Merge sort



$size = right - left + 1$

$i = l$ to m

$j = m+1$ to r

$k = 0$ to $size-1$

```

while( i <= mid && j <= right) {
    if( arr[i] < arr[j] ) {
        temp[k] = arr[i];
        i++; k++;
    }
    else {
        temp[k] = arr[j];
        j++; k++;
    }
}

```

```

while( j <= right) {
    temp[k] = arr[j];
    j++; k++;
}
while( i <= mid) {
    temp[k] = arr[i];
    i++; k++;
}

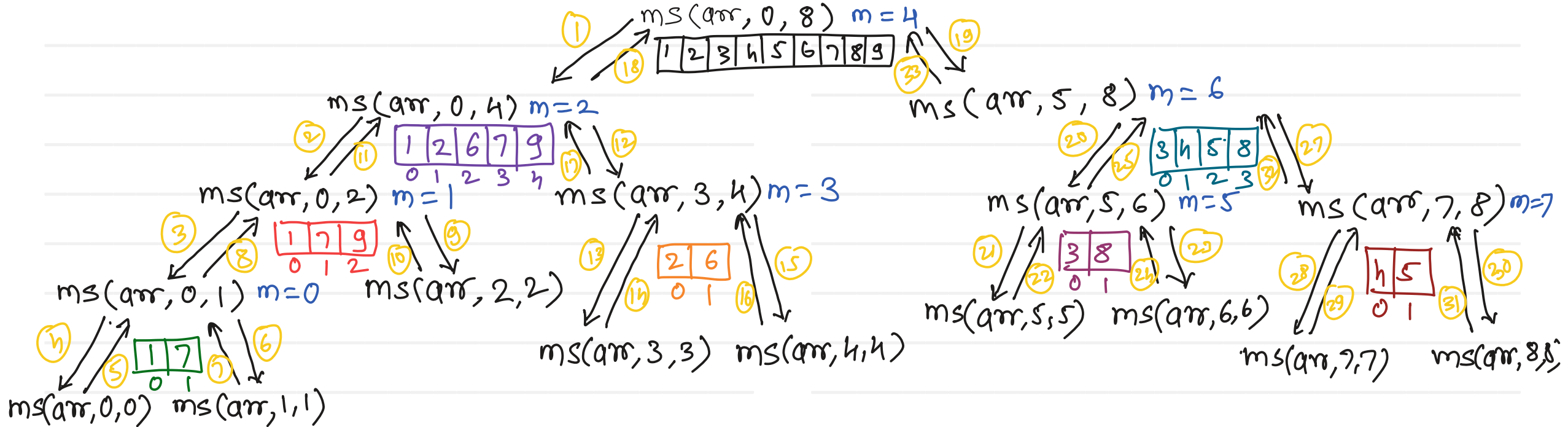
```

when left partition will finish first

when right partition will finish first

Merge sort

1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9
7	4	9	6	2	8	3	5	1
0	1	2	3	4	5	6	7	8

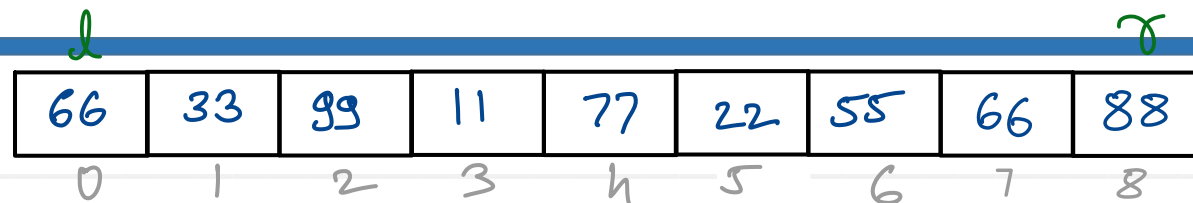


1. Select pivot/axis/reference element from array
2. Arrange lesser elements on left side of pivot
3. Arrange greater elements on right side of pivot
4. Sort left and right side of pivot again (by quick sort)

- pivot is selected by any one of the below method

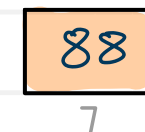
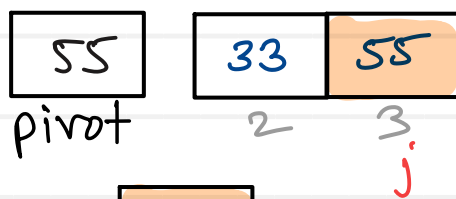
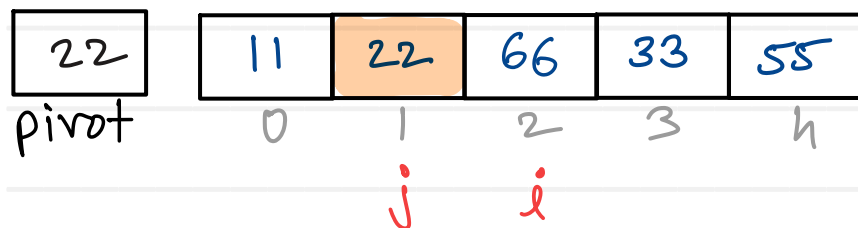
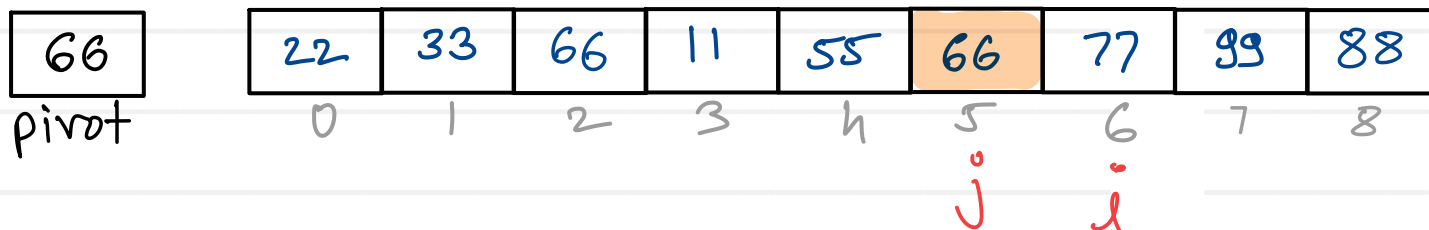
1. extreme left
2. extreme right
3. mid
4. median of three
5. median of five
6. dual pivot

Quick sort

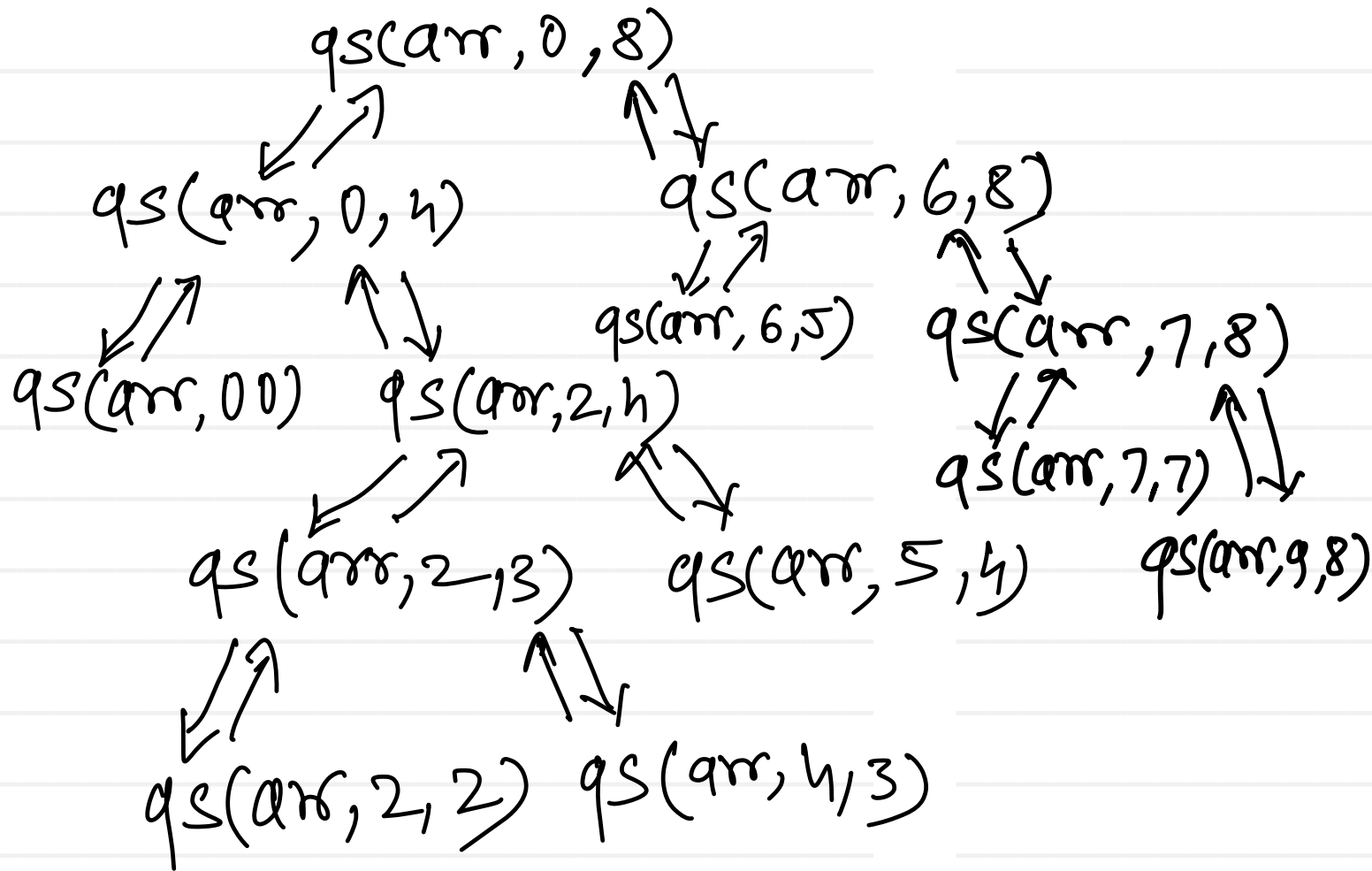


- pivot is always placed at j th index

left partition: l to $j-1$
right partition: $j+1$ to r



Quick sort



No. of levels = $\log n$

No. of comps/level = n

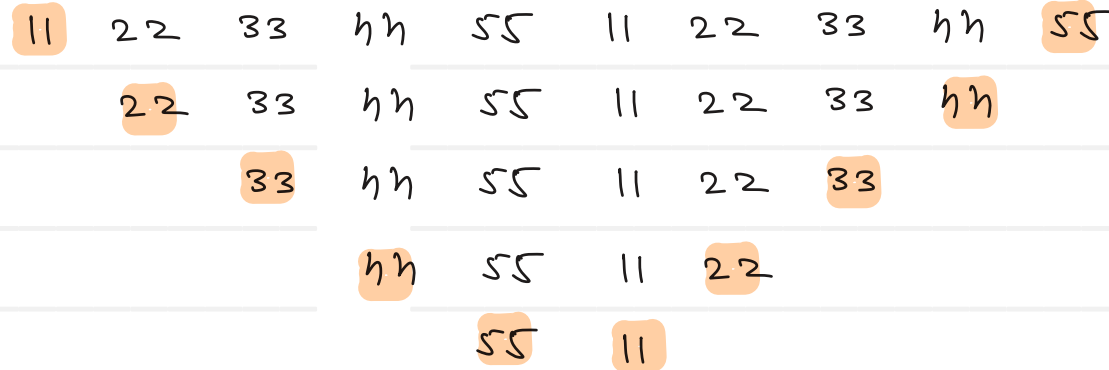
Total comps = $n \log n$

Time $\propto n \log n$

Best
Avg

$$T(n) = O(n \log n)$$

$$S(n) = O(1)$$



No. of levels $\approx n$
comps/level = n
total comps = n^2
Time $\propto n^2$

Worst

$$T(n) = O(n^2)$$

11 22 33 44 55

11 22

44 55

22

55

-time complexity is dependent on selection of pivot

	space	Best	Time Avg	Worst
selection sort	$O(1)$ in place sorting algorithm	$O(n^2)$	$O(n^2)$	$O(n^2)$
bubble sort		$O(n)$	$O(n^2)$	$O(n^2)$
insertion sort		$O(n)$	$O(n^2)$	$O(n^2)$
Heap sort		$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick sort		$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$